

Sure! Below is a **practical industry-focused table** of the **most-used programming languages** and their **core concepts/paradigms** used day-to-day in application/software development.

✓ I'm using **underlining** for core concepts like *OOP*, *Procedural*, *Functional*, etc. as you asked.

✓ **Most Used Programming Languages in IT Industry + Core Concepts (Day-to-Day)**

Programming Language	Major Use in Industry	Core Concepts / Paradigms (Underlined)	Common Day-to-Day Work
Java	Enterprise apps, Banking, Backend, Android	OOP, Procedural, Modular, Concurrent, Generic Programming	REST APIs, Microservices (Spring Boot), JDBC/JPA, Multithreading
Python	AI/ML, Automation, Backend, Scripting	OOP, Procedural, Functional, Scripting	Data processing, automation scripts, FastAPI/Django APIs, ML models
JavaScript	Web frontend + backend	Event-driven, Functional, Prototype-based OOP, Asynchronous	React/Angular UI, Node.js APIs, Promises/async-await, DOM
TypeScript	Scalable Web Apps	OOP, Functional, Static Typing, Modular	React + TS, Angular, large UI apps, typed API integration
C	OS, Embedded, Drivers	Procedural, Low-Level, Structured Programming	Memory handling, pointers, device-level development
C++	Games, High performance systems	OOP, Procedural, Generic Programming, Low-Level	STL, performance coding, real-time systems, game engines
C#	Windows apps, .NET backend	OOP, Event-driven, Functional, Concurrent	ASP.NET APIs, desktop apps, Azure cloud projects
Go (Golang)	Cloud, DevOps tools, microservices	Procedural, Concurrent, Modular	Microservices, Kubernetes services, goroutines, APIs

Kotlin	Android + Backend	OOP, Functional, Null Safety, Modular	Android apps, backend with Ktor/Spring, coroutine async
Swift	iOS development	OOP, Protocol-Oriented, Functional	iOS apps, UIKit/SwiftUI, async programming
PHP	Web development	Procedural, OOP, Scripting	Laravel apps, CMS (WordPress), APIs, server-side rendering
Ruby	Web apps	OOP, Functional, Scripting	Rails apps, backend APIs, automation scripting
SQL	Database querying	Declarative, Set-based	Queries, joins, aggregation, views, procedures, tuning
PL/SQL	Oracle enterprise systems	Procedural, Declarative, Database Programming	Stored procedures, triggers, packages (Oracle DB)
T-SQL	Microsoft SQL Server	Procedural, Declarative, Database Programming	Stored procedures, ETL, SQL Server reporting
R	Analytics / statistics	Functional, Procedural, Data-Oriented	Statistical modeling, data visualization, analysis pipelines
Shell Script (Bash)	Linux automation / DevOps	Scripting, Procedural	Deploy scripts, cron jobs, server automation
PowerShell	Windows automation	Scripting, Object-based, Procedural	Server automation, Azure scripting, IT admin scripts
Rust	Secure systems programming	Functional, Low-Level, Memory-safe, Concurrent	Secure apps, performance systems, memory-safe services

Scala	Big Data / Spark	OOP, Functional, Concurrent	Apache Spark jobs, big data pipelines, functional coding
Dart	Mobile apps (Flutter)	OOP, Event-driven, Async	Flutter apps, UI state management, APIs
HTML	Frontend basics	Markup (Declarative)	UI structure, forms, page layouts
CSS	Styling & UI design	Declarative	Styling pages, animations, responsive layouts
XML / JSON	Data formats	Declarative	API request/response, config files, integration

✔ Core Programming Paradigms Used in IT (Quick Summary)

Core Concept	Meaning	Common Languages
OOP	Classes, objects, inheritance, polymorphism	Java, C#, C++, Python, Kotlin
Procedural	Step-by-step instructions, functions	C, Python, Go, Shell
Functional	Pure functions, immutability, lambda, map/filter	JS, Python, Scala, Rust
Event-driven	Program reacts to events (click/API/message)	JS, C#, Dart
Declarative	Tell <i>what</i> to do, not <i>how</i>	SQL, HTML, CSS
Concurrent	Multiple tasks running together	Java, Go, Rust, C#
Scripting	Automation + quick programming	Python, Bash, PowerShell

If you want, I can also provide a **separate table** of:

- ✔ languages used specifically in **Banking/Finance IT**
- ✔ languages used in **DevOps / Cloud / Kubernetes**
- ✔ languages used in **AI/ML** with tools (TensorFlow, PyTorch, HuggingFace etc.)

Sure! Below is a **complete industry-level table** of the **most important Core Programming Concepts / Paradigms** used in IT day-to-day, with **underlined names** and clear **differences**.

✔ Core Concepts / Paradigms (Underlined) + Differences (Industry Use)

Core Concept / Paradigm	Meaning (Simple Definition)	Key Focus / Rule	How it differs from others	Common Examples / Where Used
Procedural Programming	Program is written as step-by-step instructions using functions	<i>Procedures/functions + sequence</i>	Everything is done in order (like a recipe)	C, Python scripts, Bash scripts
Structured Programming	Procedural with strict structure using if/else, loops	<i>No goto, structured flow</i>	Improves procedural programming by making it cleaner	C, Java, Python
Object-Oriented Programming (OOP)	Program is built around objects (data + behavior)	<i>Classes, objects, encapsulation</i>	Unlike procedural, data and functions are combined	Java, C#, C++, Python
Functional Programming	Program is built using pure functions	<i>Immutability + no side effects</i>	Unlike OOP, focus is on functions not objects	JavaScript, Scala, Haskell, Python
Declarative Programming	You describe what you want , not how to do it	<i>Result-oriented</i>	Unlike procedural, no step-by-step instructions	SQL, HTML, CSS
Imperative Programming	You tell the program how to do things step-by-step	<i>Commands & state changes</i>	Opposite of declarative	C, Java, Python
Event-Driven Programming	Program reacts to events	<i>Event loop + listeners</i>	Execution depends on events not fixed flow	JavaScript UI, Node.js, Android
Asynchronous Programming	Code runs without blocking (parallel waiting)	<i>Non-blocking execution</i>	Unlike synchronous, tasks don't wait	JS async/await, Python async

Synchronous Programming	Tasks execute one-by-one	<i>Blocking execution</i>	Opposite of async	Traditional Java, C
Concurrent Programming	Multiple tasks progress in overlapping time	<i>Threading, multitasking</i>	Unlike async: concurrent can use threads	Java threads, Go routines
Parallel Programming	Multiple tasks run at the exact same time	<i>Multi-core execution</i>	Parallel is a type of concurrency but stronger	GPU computing, HPC systems
Multithreading	Running multiple threads in a program	<i>Threads share memory</i>	A technique for concurrency	Java, C#, C++
Multiprocessing	Multiple processes run separately	<i>Separate memory per process</i>	Safer than threads, heavier resources	Python multiprocessing, OS
Generic Programming	Write code that works with any type	<i>Type parameters</i>	Enables reusable logic	Java Generics, C++ templates
Modular Programming	Break application into modules/components	<i>Reusability + separation</i>	Different from OOP (modules may not be objects)	Node modules, Java packages
Component-Based Programming	UI/logic built using independent components	<i>Reusable components</i>	Mostly used in UI frameworks	React, Angular, Vue
Prototype-based OOP	Objects inherit directly from other objects	<i>Prototype chain</i>	Different from class-based OOP	JavaScript
Logic Programming	Program is written using facts + rules	<i>Inference engine</i>	Unlike OOP/procedural, system derives answers	Prolog, AI rule engines
Data-Oriented Programming	Focus on data layout for performance	<i>Optimize memory/cache</i>	Different from OOP (no object overhead)	Game engines, HPC
Low-Level Programming	Close to hardware, manual memory	<i>Pointers, memory, CPU</i>	Opposite of high-level scripting	C, Assembly

High-Level Programming	Close to human language, abstracted	<i>Productivity over hardware details</i>	Opposite of low-level	Python, JavaScript
Scripting	Quick automation programs	<i>Fast development</i>	Used for automation, not system programming	Python, Bash, PowerShell
Reactive Programming	Programming using data streams	<i>Auto update with data changes</i>	Different from event-driven: stream-based	RxJava, Reactor, Kotlin Flow
Protocol-Oriented Programming	Design with protocols/interfaces	<i>Composition over inheritance</i>	Different from class inheritance	Swift
Aspect-Oriented Programming (AOP)	Separate cross-cutting concerns	<i>Logging, security, auditing</i>	Different: not main logic, supports clean separation	Spring AOP (Java)
Domain-Driven Design (DDD)	Design around business domains	<i>Entities, value objects, aggregates</i>	Not a coding style, but architecture/design paradigm	Banking/Enterprise apps

If you want, I can also make a **separate table** for:

✓ OOP vs Functional vs Procedural **direct comparison** with code examples (Java + Python).